Step 2: Cart, Shop page, Checkout, Mailgun email

Files to create (all inside your `my-shop` project):

```
context/CartContext.jsx
app/page.js                    (shop page,
replaces the default)
app/checkout/page.js
app/api/send-confirmation/route.js
```

Also update `app/layout.js` (see section 5) and add 2 env vars (section 6).

---

1. `context/CartContext.jsx`: cart state shared by every page

```
"use client";

import { createContext, useContext,
useEffect, useState } from "react";

// A context lets any component read/change
the cart without passing props around
const CartContext = createContext(null);
```

```
export function CartProvider({ children })
{
  // cart = array of { id, name, price,
image_url, quantity }
  const [cart, setCart] = useState([]);

  // Load the saved cart from localStorage
once, when the app first opens
  useEffect(() => {
    try {
      const saved =
localStorage.getItem("cart");
      if (saved)
setCart(JSON.parse(saved));
    } catch {}
  }, []);

  // Save the cart every time it changes,
so a refresh doesn't wipe it
  useEffect(() => {
    localStorage.setItem("cart",
JSON.stringify(cart));
  }, [cart]);

  // Add a product: if already in cart,
increase quantity; otherwise add it with
quantity 1
  function addToCart(product) {
    setCart((prev) => {
      const existing = prev.find((i) =>
i.id === product.id);
      if (existing) {
        return prev.map((i) =>
```

```
          i.id === product.id ? { ...i,
quantity: i.quantity + 1 } : i
        );
      }
      return [...prev, { ...product,
quantity: 1 }];
    });
  }

  // Change quantity by +1 or -1; remove
the item if it reaches 0
  function changeQty(id, delta) {
    setCart((prev) =>
      prev
        .map((i) => (i.id === id ? { ...i,
quantity: i.quantity + delta } : i))
        .filter((i) => i.quantity > 0)
    );
  }

  function clearCart() {
    setCart([]);
  }

  // Derived values, recalculated on every
render
  const total = cart.reduce((sum, i) => sum
+ i.price * i.quantity, 0);
  const count = cart.reduce((sum, i) => sum
+ i.quantity, 0);

  return (
    <CartContext.Provider
      value={{ cart, addToCart, changeQty,
```

```
clearCart, total, count }}
    >
      {children}
    </CartContext.Provider>
  );
}

// Shortcut hook: const { cart, addToCart }
= useCart();
export const useCart = () =>
useContext(CartContext);
```

Line by line:

- `createContext` makes a shared "box" for cart data; `CartProvider` fills it.
- The first `useEffect` (empty `[]`) runs once and restores the cart from `localStorage`.
- The second `useEffect` (depends on `[cart]`) runs after every cart change and saves it.
- `addToCart` uses `setCart(prev => ...)` so it always works from the latest cart.
- `changeQty` adds `delta` (+1 or -1), then `.filter` drops items whose quantity hit 0.
- `total` and `count` use `reduce` to sum up price x quantity and total items.

---

2. `app/page.js`: shop page (product grid)

```
"use client";

import { useEffect, useState } from
"react";
import { supabase } from "@/lib/supabase";
import { useCart } from
"@/context/CartContext";

export default function Shop() {
  const [products, setProducts] =
useState([]);
  const [loading, setLoading] =
useState(true);
  const { addToCart } = useCart();

  // Fetch all products from Supabase when
the page loads
  useEffect(() => {
    supabase
      .from("products")
      .select("*")
      .order("created_at")
      .then(({ data, error }) => {
        if (!error) setProducts(data);
        setLoading(false);
      });
  }, []);

  if (loading) return <p>Loading
```

```
products...</p>;

  return (
    <div className="grid grid-cols-2 gap-4
md:grid-cols-3">
      {products.map((p) => (
        <div key={p.id} className="rounded
border p-3">
          <img src={p.image_url} alt=
{p.name} className="mb-2 w-full rounded" />
          <h2 className="font-semibold">
{p.name}</h2>
          <p className="text-sm text-gray-
600">{p.description}</p>
          <p className="my-2 font-bold">₦
{Number(p.price).toLocaleString()}</p>
          <button
            onClick={() => addToCart(p)}
            className="w-full rounded bg-
black py-2 text-white"
          >
            Add to cart
          </button>
        </div>
      ))}
    </div>
  );
}
```

Line by line:

- `supabase.from("products").select("*")` = `SELECT * FROM products`.

- `.order("created_at")` keeps a stable order.
- `Number(p.price)` is needed because Supabase returns `numeric` columns as strings.
- `toLocaleString()` adds commas (₦32,000).

---

3. `app/checkout/page.js`: cart review, login check, place order

```
"use client";

import { useEffect, useState } from
"react";
import { supabase } from "@/lib/supabase";
import { useCart } from
"@/context/CartContext";

export default function Checkout() {
  const { cart, changeQty, clearCart, total
} = useCart();
  const [user, setUser] = useState(null);
  const [checking, setChecking] =
useState(true); // true until we know login
status
  const [placing, setPlacing] =
useState(false);
  const [orderId, setOrderId] =
useState(null);   // set after success
  const [error, setError] = useState("");
```

```
  // Find out who is logged in
  useEffect(() => {
    supabase.auth.getSession().then(({ data
}) => {
      setUser(data.session?.user ?? null);
      setChecking(false);
    });
  }, []);

  async function login() {
    await supabase.auth.signInWithOAuth({
      provider: "google",
      // After Google login, come back to
the checkout page
      options: { redirectTo:
window.location.origin + "/checkout" },
    });
  }

  async function placeOrder() {
    setPlacing(true);
    setError("");
    try {
      // 1. Insert the order row;
.select().single() returns the new row (we
need its id)
      const { data: order, error: orderErr
} = await supabase
        .from("orders")
        .insert({
          user_id: user.id,
          email: user.email,
          total,
          status: "paid", // no real
```

```
payment in this task, so we mark it paid
        })
        .select()
        .single();
      if (orderErr) throw orderErr;

      // 2. Insert one row per cart item,
all linked to the order id
      const items = cart.map((i) => ({
        order_id: order.id,
        product_id: i.id,
        quantity: i.quantity,
        price: i.price,
      }));
      const { error: itemsErr } = await
supabase.from("order_items").insert(items);
      if (itemsErr) throw itemsErr;

      // 3. Ask our server route to send
the email.
      //    We pass the login token so the
server can prove who is asking.
      const { data: sess } = await
supabase.auth.getSession();
      await fetch("/api/send-confirmation",
{
        method: "POST",
        headers: {
          "Content-Type":
"application/json",
          Authorization: `Bearer
${sess.session.access_token}`,
        },
        body: JSON.stringify({ orderId:
```

```
order.id }),
      });

      // 4. Success: empty the cart and
show the confirmation screen
      clearCart();
      setOrderId(order.id);
    } catch (e) {
      setError(e.message || "Something went
wrong. Please try again.");
    } finally {
      setPlacing(false);
    }
  }

  if (checking) return <p>Loading...</p>;

  // Success screen
  if (orderId)
    return (
      <div className="text-center">
        <h1 className="text-2xl font-
bold">Order confirmed 🎉</h1>
        <p className="mt-2">Order ID:
{orderId}</p>
        <p>A confirmation email has been
sent to {user.email}.</p>
        <a href="/" className="mt-4 inline-
block underline">Keep shopping</a>
      </div>
    );

  if (cart.length === 0)
    return <p>Your cart is empty. <a
```

```
href="/" className="underline">Go
shopping</a></p>;

  return (
    <div>
      <h1 className="mb-4 text-2xl font-
bold">Checkout</h1>

      {cart.map((i) => (
        <div key={i.id} className="flex
items-center justify-between border-b py-
2">
          <span>{i.name}</span>
          <div className="flex items-center
gap-2">
            <button onClick={() =>
changeQty(i.id, -1)} className="rounded
border px-2">-</button>
            <span>{i.quantity}</span>
            <button onClick={() =>
changeQty(i.id, 1)} className="rounded
border px-2">+</button>
          </div>
          <span>₦{(i.price *
i.quantity).toLocaleString()}</span>
        </div>
      ))}

      <p className="my-4 text-right text-lg
font-bold">
        Total: ₦{total.toLocaleString()}
      </p>

      {error && <p className="mb-2 text-
```

```
red-600">{error}</p>}

      {user ? (
        <button
          onClick={placeOrder}
          disabled={placing}
          className="w-full rounded bg-
black py-3 text-white disabled:opacity-50"
        >
          {placing ? "Placing order..." :
"Place order"}
        </button>
      ) : (
        <button onClick={login}
className="w-full rounded bg-black py-3
text-white">
          Log in with Google to checkout
        </button>
      )}
      <p className="mt-2 text-center text-
xs text-gray-500">
        Demo shop: no real payment is
taken.
      </p>
    </div>
  );
}
```

Line by line:

- The order is saved first, then the items. Row Level Security allows this because `user_id` equals the logged-in user.

- `.select().single()` after `insert` returns the created order, which gives us `order.id`.
- The email call comes last, so a failed email never blocks a saved order.
- `finally` always turns the "placing" state off, even after an error.

---

4. `app/api/send-confirmation/route.js`: server-only Mailgun sender

```
import { createClient } from
"@supabase/supabase-js";

export async function POST(req) {
  try {
    // 1. Read the login token and the
order id sent by the browser
    const token =
(req.headers.get("authorization") ||
"").replace("Bearer ", "");
    const { orderId } = await req.json();
    if (!token || !orderId) {
      return Response.json({ error:
"Missing data" }, { status: 400 });
    }

    // 2. Make a Supabase client that acts
AS THIS USER (so RLS applies).
    //    If the order is not theirs, the
```

```
query returns nothing.
    const supabase = createClient(
      process.env.NEXT_PUBLIC_SUPABASE_URL,

process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY,
      { global: { headers: { Authorization:
`Bearer ${token}` } } }
    );

    // 3. Load the order together with its
items and each product's name
    const { data: order, error } = await
supabase
      .from("orders")
      .select("id, email, total,
order_items(quantity, price,
products(name))")
      .eq("id", orderId)
      .single();
    if (error || !order) {
      return Response.json({ error: "Order
not found" }, { status: 404 });
    }

    // 4. Build the email body
    const lines = order.order_items
      .map((i) => `${i.quantity} x
${i.products.name} - ₦${(i.price *
i.quantity).toLocaleString()}`)
      .join("\n");
    const text =
      `Thank you for your order!\n\nOrder
ID: ${order.id}\n\n${lines}\n\n` +
      `Total: ₦
```

```
${Number(order.total).toLocaleString()}`;

    // 5. Send through Mailgun's HTTP API
(key stays on the server)
    const form = new URLSearchParams();
    form.append("from", `My Shop
<postmaster@${process.env.MAILGUN_DOMAIN}>`
);
    form.append("to", order.email);
    form.append("subject", `Order
confirmation #${order.id.slice(0, 8)}`);
    form.append("text", text);

    const res = await fetch(

`https://api.mailgun.net/v3/${process.env.M
AILGUN_DOMAIN}/messages`,
      {
        method: "POST",
        headers: {
          Authorization:
            "Basic " +
Buffer.from(`api:${process.env.MAILGUN_API_
KEY}`).toString("base64"),
        },
        body: form,
      }
    );

    if (!res.ok) {
      const msg = await res.text();
      return Response.json({ error: msg },
{ status: 502 });
    }
```

```
    return Response.json({ ok: true });
  } catch (e) {
    return Response.json({ error: e.message
}, { status: 500 });
  }
}
```

Line by line:

- This file runs only on the server, so `MAILGUN_API_KEY` is never exposed to the browser.
- Using the user's token means nobody can trigger emails for someone else's order.
- `order_items(... products(name))` is a Supabase joined query: order, then its items, then each product name.
- Mailgun uses Basic auth with the username `api` and your API key as the password.
- If your Mailgun account is in the EU region, change the URL host to `api.eu.mailgun.net`.

---

5. `app/layout.js`: wrap the app in the cart and add a cart link

```
import "./globals.css";
import AuthButton from
"@/components/AuthButton";
```

```
import { CartProvider } from
"@/context/CartContext";
import CartLink from
"@/components/CartLink";

export const metadata = { title: "My Shop"
};

export default function RootLayout({
children }) {
  return (
    <html lang="en">
      <body>
        <CartProvider>
          <header className="flex items-
center justify-between border-b p-4">
            <a href="/" className="text-xl
font-bold">My Shop</a>
            <div className="flex items-
center gap-4">
              <CartLink />
              <AuthButton />
            </div>
          </header>
          <main className="mx-auto max-w-
5xl p-4">{children}</main>
        </CartProvider>
      </body>
    </html>
  );
}
```

`components/CartLink.jsx`:

```
"use client";
import { useCart } from
"@/context/CartContext";

// Shows "Cart (3)" and links to checkout
export default function CartLink() {
  const { count } = useCart();
  return <a href="/checkout"
className="underline">Cart ({count})</a>;
}
```

---

6. Environment variables

Add to `.env.local` (and later to Vercel -> Settings -> Environment Variables):

```
MAILGUN_API_KEY=your-mailgun-private-api-
key
MAILGUN_DOMAIN=sandboxXXXXXXXX.mailgun.org
```

No `NEXT_PUBLIC_` prefix, so they stay server-only.

Mailgun sandbox reminder: Mailgun dashboard -> Sending -> your sandbox domain -> Authorized Recipients. Add the email you'll test with and click the verification link Mailgun sends. Check spam if nothing arrives.

Test checklist

1. `npm run dev`, open `http://localhost:3000`.
2. Products load, "Add to cart" updates the header count.
3. Open the cart, log in with Google, click "Place order".
4. Confirmation screen appears, the order shows in Supabase -> Table Editor -> `orders`, and the email arrives.